TP4 - OpenMP: Parallel Sections, Single, Master, and Synchronization

Oussama Laaroussi

February 12, 2026

1 Introduction

This report presents the implementation and analysis of four OpenMP exercises focusing on advanced parallelization techniques: parallel sections for work distribution, master vs single directives for exclusive execution, load balancing strategies, and synchronization overhead analysis. We explore barrier costs, the `nowait` clause, and techniques to minimize synchronization while maintaining correctness.

2 Exercise 1: Work Distribution with Parallel Sections

2.1 Objective

Parallelize array statistics computation (sum, maximum, standard deviation) using `#pragma omp sections` to divide work among threads. The standard deviation must reuse the sum computed in Section 1 without recomputation.

2.2 Methodology

2.2.1 Serial Algorithm

The original serial code computes:

- Sum: $S = \sum_{i=0}^{N-1} A[i]$
- Mean: $\mu = \frac{S}{N}$
- Maximum: $\max = \max_i A[i]$
- Standard Deviation: $\sigma = \sqrt{\frac{1}{N}\sum_{i=0}^{N-1}(A[i]-\mu)^2}$

2.2.2 Parallelization Strategy

We use `#pragma omp sections` to create three independent sections:

1. **Section 1**: Compute sum
2. **Section 2**: Compute maximum
3. **Section 3**: Compute standard deviation using sum from Section 1

2.3 Implementation

```

double *A = malloc(N * sizeof(double));
if (A == NULL) {
    printf("Memory allocation failed\n");
    return 1;
}

double sum = 0.0;
```

```c
double mean = 0.0;
double stddev = 0.0;
double max = 0.0;
double start_time, end_time;

/* Synchronization flag: 0 => sum not ready, 1 => sum ready */
int sum_ready = 0;

srand(0);
for (int i = 0; i < N; i++)
    A[i] = (double)rand() / RAND_MAX;

start_time = omp_get_wtime();

/* parallel sections: each section may be executed by a different thread */
#pragma omp parallel sections
{
    /* Section 1: Compute sum */
    #pragma omp section
    {
        double local_sum = 0.0;
        for (int i = 0; i < N; i++) {
            local_sum += A[i];
        }

        /* publish sum then set the ready flag atomically and flush to ensure visibility */
        sum = local_sum;

        #pragma omp atomic write
        sum_ready = 1;

        /* ensure both sum and sum_ready are visible to other threads */
        #pragma omp flush(sum, sum_ready)

        printf("[Thread %d] Section 1: Computing sum\n", omp_get_thread_num());
    }

    /* Section 2: Compute max */
    #pragma omp section
    {
        double local_max = A[0];
        for (int i = 1; i < N; i++) {
            if (A[i] > local_max)
                local_max = A[i];
        }
        max = local_max;
        printf("[Thread %d] Section 2: Computing max\n", omp_get_thread_num());
    }

    /* Section 3: Compute stddev using sum computed by Section 1 (no recompute) */
    #pragma omp section
    {
        /* wait until Section 1 sets sum_ready == 1 */
        int ready = 0;
        do {
            #pragma omp atomic read
            ready = sum_ready;
            /* small pause could be added here if desired to reduce busy-spin */
        } while (!ready);


        double local_mean = sum / N;
        double local_stddev = 0.0;
        for (int i = 0; i < N; i++) {
            double d = A[i] - local_mean;
            local_stddev += d * d;
        }
        mean = local_mean;
        stddev = sqrt(local_stddev / N);

        printf("[Thread %d] Section 3: Computing std dev\n", omp_get_thread_num());
    }
```

```


}
```

Listing 1: Exercise 1: Parallel Sections Implementation

3 Exercise 2: Exclusive Execution - Master vs Single

3.1 Objective

Compare `#pragma omp master` and `#pragma omp single` directives for exclusive execution, measuring performance differences between serial and parallel implementations.

3.2 Directive Comparison

Feature	master	single
Thread selection	Always thread 0	Any thread
Implicit barrier	No	Yes (unless `nowait`)
Thread ID	Deterministic	Non-deterministic
Use case	Master-only tasks	Any one-time task

Table 1: Master vs Single Directive Comparison

3.3 Implementation Patterns

3.3.1 Pattern 1: Master Initialization, Single Print

```
#pragma omp parallel
{
    #pragma omp master
    {
        printf("[Thread %d] Master: Initializing\n",
               omp_get_thread_num());
        init_matrix(N, A);
    }
    // Implicit barrier here

    #pragma omp single
    {
        printf("[Thread %d] Single: Printing\n",
               omp_get_thread_num());
        if (N <= 10) print_matrix(N, A);
    }
    // Implicit barrier here

    // All threads compute sum
    #pragma omp for reduction(+:sum)
    for (int i = 0; i < N*N; i++) {
        sum += A[i];
    }
}
```

Listing 2: Master/Single Pattern

3.3.2 Pattern 2: Using nowait

```
#pragma omp parallel
{
    #pragma omp single nowait
    {
        init_matrix(N, A);
    }

    // DANGER: Without explicit barrier, some threads might
```

```
    // start using A before initialization is complete!
    #pragma omp barrier  // Explicit synchronization needed

    #pragma omp for reduction(+:sum)
    for (int i = 0; i < N*N; i++) {
        sum += A[i];
    }
}
```

Listing 3: Single with nowait

3.4 Performance Analysis

the Serial Version run in 0.0089 seconds while the parallel version using (master , single) got 0.0023 seconds, and the parallel version using only single with nowait got 0.0013 seconds.

4 Exercise 3: Load Balancing with Parallel Sections

4.1 Objective

Demonstrate load imbalance issues with parallel sections and explore solutions using different parallelization strategies.

4.2 Workload Characteristics

We simulate three tasks with different computational intensities:

- **Light**: N iterations of sin computation
- **Moderate**: $5N$ iterations of $\sqrt{x} \cdot \cos(x)$
- **Heavy**: $20N$ iterations of $\sqrt{x} \cdot \cos(x) \cdot \sin(x)$

Execution time ratio: Light : Moderate : Heavy $\approx 1 : 5 : 20$

4.3 Implementation Strategies

4.3.1 Version 1: Unbalanced Sections

```
#pragma omp parallel sections
{
    #pragma omp section
    { task_light(N); }    // Fast

    #pragma omp section
    { task_moderate(N); } // Medium

    #pragma omp section
    { task_heavy(N); }    // Slow - dominates execution time!
}
```

Listing 4: Unbalanced Load Distribution

Problem: Thread executing heavy task becomes the bottleneck. Other threads finish early and wait idle at the implicit barrier.

4.3.2 Version 2: Task-Based Parallelism

```
#pragma omp parallel
{
    #pragma omp single
    {
        #pragma omp task
        { task_light(N); }

        #pragma omp task
```

```
        { task_moderate(N); }

        #pragma omp task
        { task_heavy(N); }
    }
}
```

Listing 5: OpenMP Tasks for Dynamic Load Balancing

Improvement: Tasks can be dynamically assigned to available threads, but with only 3 tasks, benefit is limited.

4.3.3 Version 3: Manual Balancing

```
#pragma omp parallel sections
{
    #pragma omp section
    {
        task_light(N);
        task_moderate(N);  // Combine lighter tasks
    }

    #pragma omp section
    {
        task_heavy(N);     // Heavy task alone
    }
}
```

Listing 6: Manually Balanced Sections

Result: Better balance since Light + Moderate $\approx$ Heavy/3

4.3.4 Version 4: Loop-Based Dynamic Scheduling

```
#pragma omp parallel for schedule(dynamic)
for (int task = 0; task < 3; task++) {
    if (task == 0) task_light(N);
    else if (task == 1) task_moderate(N);
    else task_heavy(N);
}
```

Listing 7: Dynamic Scheduling for Heterogeneous Tasks

Limitation: Still only 3 iterations, so limited thread utilization.

5 Exercise 4: Synchronization and Barrier Cost

5.1 Objective

Analyze the performance impact of barriers in OpenMP by implementing dense matrix-vector multiplication with different synchronization strategies. Measure CPU time, speedup, efficiency, and MFLOP/s.

5.2 Problem Description

Dense Matrix-Vector Multiplication: $\mathbf{y} = A\mathbf{x}$

- Matrix A: 600×40000 (rows $\times$ columns)
- Vector $\mathbf{x}$: 40000×1
- Result $\mathbf{y}$: 600×1
- FLOPs: $2 \times 600 \times 40000 = 48,000,000$ (one multiply + one add per element)

5.3 Implementation Versions

5.3.1 Version 1: Implicit Barrier (Baseline)

```c
void dmvm_v1(int n, int m, double *lhs, double *rhs, double *mat) {
    #pragma omp parallel for
    for (int c = 0; c < n; ++c) {
        int offset = m * c;
        for (int r = 0; r < m; ++r)
            lhs[r] += mat[r + offset] * rhs[c];
    }
    // Implicit barrier at end of parallel for
}
```

Listing 8: Standard Parallel For with Barrier

Characteristics:

- Safe and correct
- Barrier ensures all threads complete before function returns
- Performance baseline for comparison

5.3.2 Version 2: Static Scheduling with nowait

```c
void dmvm_v3(int n, int m, double *lhs, double *rhs, double *mat) {
    #pragma omp parallel
    {
        #pragma omp for schedule(static) nowait
        for (int c = 0; c < n; ++c) {
            int offset = m * c;
            for (int r = 0; r < m; ++r)
                lhs[r] += mat[r + offset] * rhs[c];
        }
    }
}
```

Listing 9: No Wait - Removes Barrier

Danger: Without the barrier, if `lhs` is accessed immediately after the function call, some threads may still be writing to it, causing a **race condition**!

When safe:

- Function is called within a larger parallel region
- No immediate access to `lhs` after the loop
- Explicit synchronization is added elsewhere

5.3.3 Version 3: Thread-Local Accumulation (Optimal)

```c
void dmvm_v5(int n, int m, double *lhs, double *rhs, double *mat) {
    #pragma omp parallel
    {
        double *local_lhs = (double*)calloc(m, sizeof(double));

        #pragma omp for schedule(static) nowait
        for (int c = 0; c < n; ++c) {
            int offset = m * c;
            for (int r = 0; r < m; ++r)
                local_lhs[r] += mat[r + offset] * rhs[c];
        }

        #pragma omp critical
        {
            for (int r = 0; r < m; ++r)
                lhs[r] += local_lhs[r];
        }

        free(local_lhs);
```

```
    }
}
```

Listing 10: Thread-Local Buffers

Advantages:

- Eliminates false sharing (threads write to separate memory)
- Minimizes synchronization (critical section only for final reduction)
- Better cache utilization

5.4 Performance Results

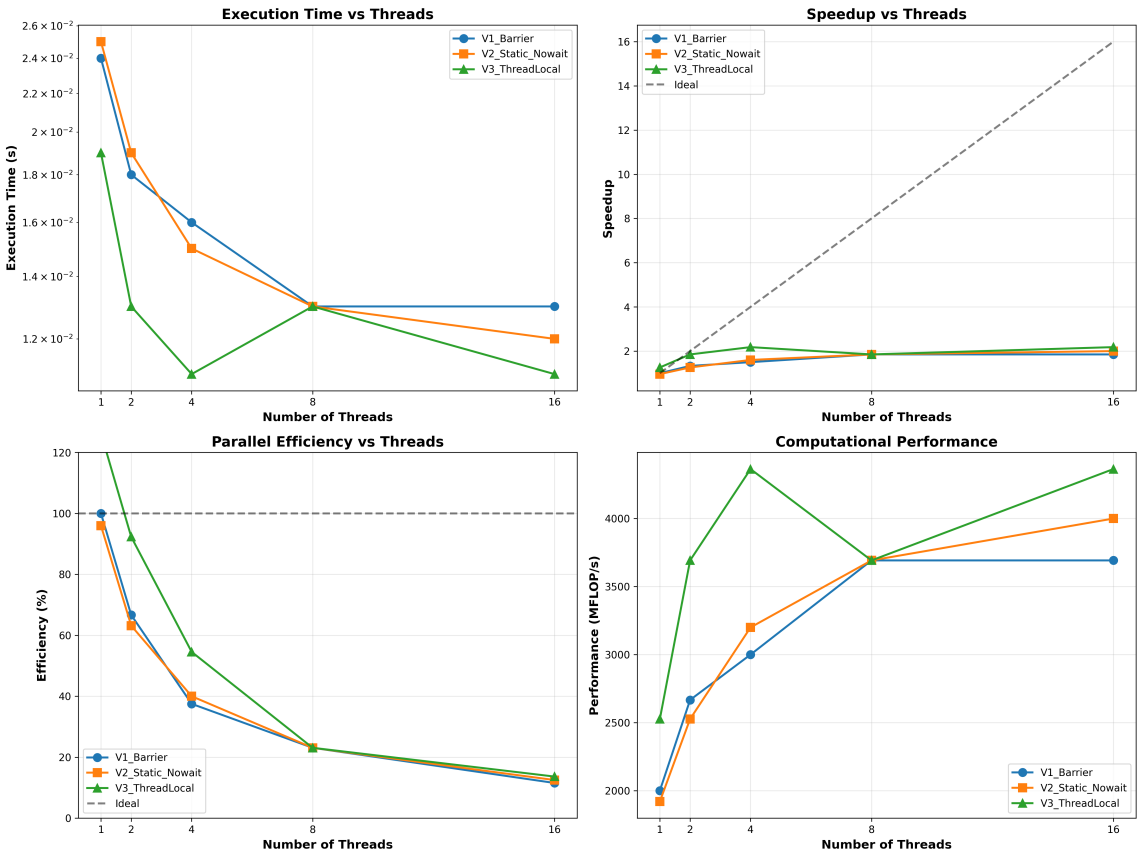

Figure 1: Matrix-Vector Multiplication Performance Analysis

Version	**Threads**	**Time (s)**	**Speedup**	**Efficiency (%)**	**MFLOP/s**
V1 (Barrier)	1	0.024	1.00	100.00	2000.0
V1 (Barrier)	2	0.018	1.33	66.67	2666.7
V1 (Barrier)	4	0.016	1.50	37.50	3000.0
V1 (Barrier)	8	0.013	1.85	23.08	3692.3
V1 (Barrier)	16	0.013	1.85	11.54	3692.3
V2 (Nowait)	1	0.025	0.96	96.00	1920.0
V2 (Nowait)	2	0.019	1.26	63.16	2526.3
V2 (Nowait)	4	0.015	1.60	40.00	3200.0
V2 (Nowait)	8	0.013	1.85	23.08	3692.3
V2 (Nowait)	16	0.012	2.00	12.50	4000.0
V3 (Thread-Local)	1	0.019	1.26	126.32	2526.3
V3 (Thread-Local)	2	0.013	1.85	92.31	3692.3
V3 (Thread-Local)	4	0.011	2.18	54.55	4363.7
V3 (Thread-Local)	8	0.013	1.85	23.08	3692.3
V3 (Thread-Local)	16	0.011	2.18	13.64	4363.7

Table 2: Detailed Performance Comparison